

FITFINDER

A Unified Gym Management Platform

CAPSTONE DEFENSE — SYSTEM DOCUMENTATION

Full-Stack Architecture, Feature & Flow Documentation

Next.js 15 · Express 5 · PostgreSQL (Neon) · Socket.IO · Prisma

Roles: Platform Admin · Gym Owner · Front Desk Clerk · Gymer

Payments: Xendit (GCash) · AI: Google Gemini

Table of Contents

1. **1.** System Overview & Problem Statement
2. **2.** Technology Stack
3. **3.** Architecture
4. **4.** Backend — Project Structure
5. **5.** Frontend — Project Structure
6. **6.** Database Design (20 Models, 8 Enums)
7. **7.** REST API Reference (105 Endpoints)
8. **8.** Authentication & Security Architecture
9. **9.** Membership & Payment Flow
10. **10.** Messaging Flow
11. **11.** Real-Time Layer (Socket.IO)
12. **12.** Notification System
13. **13.** AI Fitness Assistant
14. **14.** Attendance & Reporting
15. **15.** Role-Based Access Control
16. **16.** Deployment & DevOps
17. **17.** Design Decisions & Trade-offs
18. **18.** Testing & Known Limitations

1. System Overview & Problem Statement

1.1 Problem

Gym management in the Philippines is fragmented. Independent gym owners track memberships, walk-in sales, coach schedules, and daily revenue in paper notebooks or disconnected spreadsheets. Front desk clerks lack a fast way to register walk-in members and record OTC payments. Members have no reliable way to check their membership status, book coaches, or receive reminders before their subscription expires. Platform operators have no consolidated view of revenue or account activity.

1.2 Solution

FitFinder is a web-based multi-tenant gym management platform that consolidates the full lifecycle of a gym business — from gym discovery and membership purchase to daily attendance, sales reporting, and platform-level analytics — behind a single authenticated web application with four role-specific dashboards.

1.3 Core Capabilities

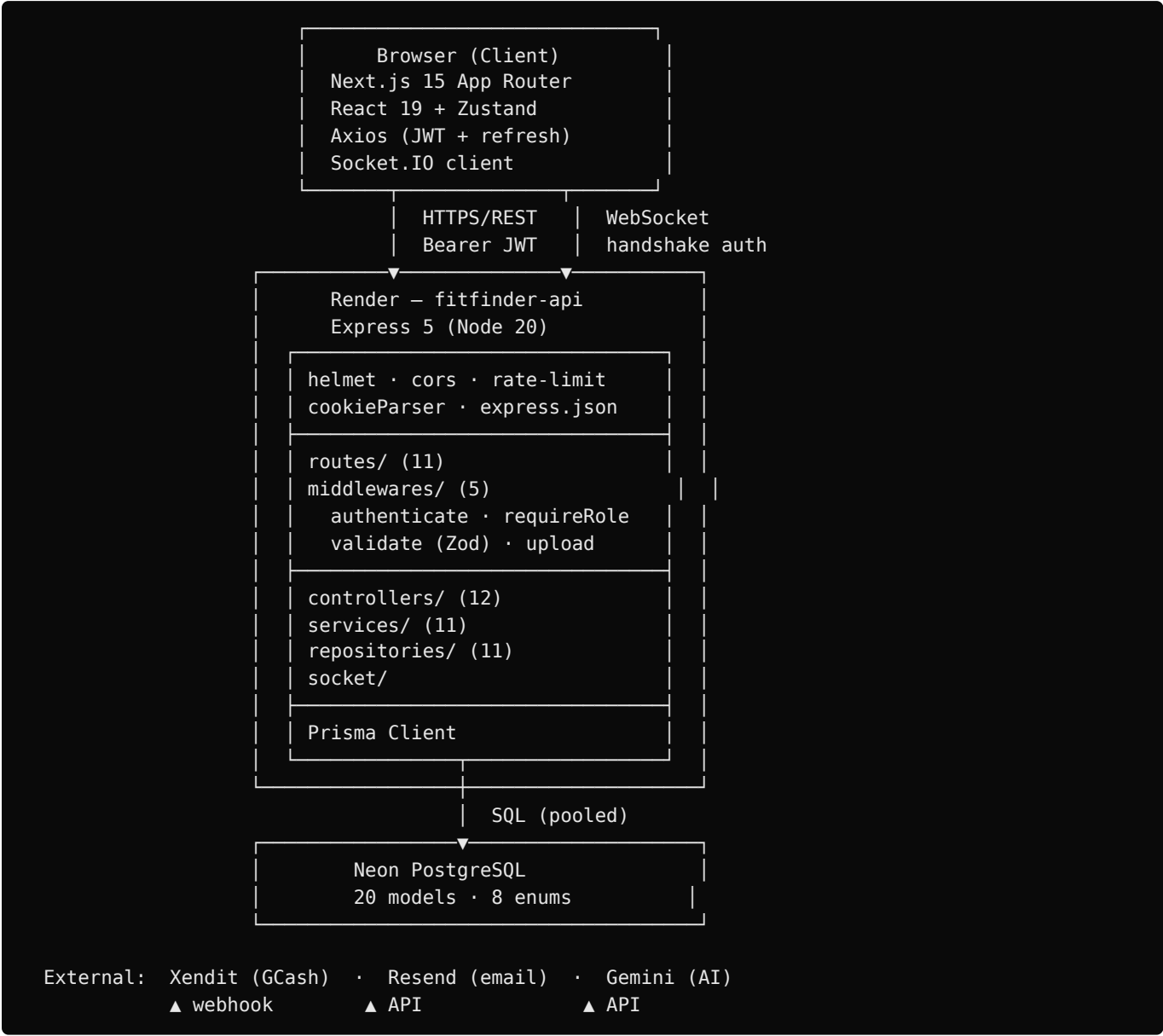
Capability	Description
Multi-role authentication	Four distinct roles (Admin, Owner, Clerk, Gymer) with JWT-based sessions, email verification, and role-gated routing
Gym discovery	Public catalog of active gyms with plans, coaches, equipment, exercises, and shop listings
Membership lifecycle	Online (GCash via Xendit) and walk-in (OTC cash) purchase, approval queue, activation, renewal, expiry
Cashless payments	Xendit integration with per-gym API keys, webhook confirmation, and owner-controlled publish toggle
Attendance tracking	Manual check-in/check-out for members and walk-in visitors, live "Active Now" dashboard
Sales reporting	Clerk daily close-out, immutable daily sales reports, owner receipts, admin platform revenue
Cross-role messaging	Real-time 1:1 direct messaging between any two users regardless of role
Notification system	Persistent inbox with dedupe keys, scheduled expiry reminders, live Socket.IO delivery
AI fitness assistant	Domain-gated Gemini integration that answers only fitness/gym questions
Platform administration	Account management, gym approval, analytics, revenue dashboards

2. Technology Stack

Layer	Technology	Version	Rationale
Frontend framework	Next.js (App Router)	^15.3.4	File-based routing where folder structure defines URLs; React Server Components; mature build optimization
UI runtime	React	^19.1.0	Component model with hooks for stateful dashboard panels
Styling	Tailwind CSS v4	^4.x	Utility-first CSS; dark theme implemented via utility classes and theme-aligned spacing tokens
Client state	Zustand	^5.0.14	Lightweight domain stores with persistence middleware; avoids Redux boilerplate
HTTP client	Axios	^1.18.1	Request/response interceptors enable the automatic 401 → refresh → retry pipeline
Realtime	Socket.IO client	^4.8.3	Rooms, reconnection, and handshake auth over WebSocket
Forms	React Hook Form + Zod	^7.81 / ^4.4	Schema-driven validation shared conceptually with backend Zod schemas
Backend runtime	Express	^5.1.0	Minimal, well-understood HTTP framework; middleware model suits auth/role gates
Language	TypeScript	^5.8	Compile-time safety across both tiers; Node 20 runtime
ORM	Prisma	^6.9	Type-safe client generated from schema; single source of truth for DB models
Database	PostgreSQL (Neon)	—	Managed serverless Postgres; pooled connections suit an intermittent free-tier deployment
Auth	jsonwebtoken + bcryptjs	^9.0 / ^3.0	Stateless JWT access tokens; bcrypt at 12 rounds for password hashing
Payments	Xendit	^6.25	GCash checkout for the Philippine market; webhook-driven confirmation
AI	Google Gemini	^0.24	Fitness-domain Q&A assistant with prompt validation and domain gating
Email	Resend	^6.25	Transactional delivery for verification codes and password-reset OTPs
Security	helmet, cors, express-rate-limit	^8.1 / ^2.8 / ^7.5	Security headers, origin allowlisting, brute-force throttling
Hosting	Render	free plan	Blueprint-driven deployment via <code>render.yaml</code>

3. Architecture

3.1 System Context



3.2 Layered Backend Design

Requests flow through a strict four-stage pipeline. Each stage has a single responsibility, which keeps concerns separable and independently testable.

HTTP Request



```
MIDDLEWARE  middlewares/  
helmet()      security headers  
cors()        origin allowlist + credentials: true  
cookieParser  read refreshToken cookie  
rateLimit     30 req / 15 min on credential routes  
authenticate  Bearer JWT → req.userId / req.userRole  
requireRole   role gate (OWNER / CLERK / ADMIN)  
validate      Zod schema → typed req.body
```



```
ROUTES  routes/  (11 files, 105 endpoints)  
Declares paths, method, middleware chain, handler  
Stable /api/* mounts defined in app.ts
```



```
CONTROLLERS  controllers/  (12 files)  
Parse request → call service → shape response  
Uniform envelope: { success, message, data, errors }
```



```
SERVICES  services/  (11 domain folders)  
Business logic, transactions, external integrations  
auth · membership · payment · subscription ·  
attendance · notification · messaging(ai) · realtime  
admin · email · gym
```



```
REPOSITORIES  repositories/  (11 classes)  
Prisma queries only – no business logic  
Every method accepts optional tx for $transaction
```



Neon PostgreSQL

Design note. Every repository method accepts an optional `tx` transaction client. This lets services compose multi-table writes atomically (for example, approving a walk-in membership touches `walk_in_approvals`, `gym_memberships`, and the plan subscriber counter) without the repository layer leaking transaction concerns into controllers.

3.3 Frontend Architecture

```
app/
├─ layout.tsx           Root: fonts, globals.css
├─ page.tsx             Landing → redirect if authed
├─ (auth)/              login · register · verify · reset
├─ landing-page/        Marketing page
├─ dashboard/
│   └─ layout.tsx       AUTH GATE + role redirect
│                       + realtime hooks mount
│   └─ admin/           6 panels
│   └─ owner/           14 panels + OwnerSidebar
│   └─ clerk/           5 panels + ClerkSidebar
│   └─ user/            Gymer: discover, join, AI, msgs
└─
```

Cross-cutting:

components/ui/	Button, Input, Label, Alert, Checkbox
components/features/	Landing, auth shell, create-gym shell
components/notifications/	NotificationBell
hooks/	(12) Socket listeners + list sync
stores/	(29) Zustand domain stores
lib/	(11) api.ts · socket.ts · media · roles

3.4 Dashboard Auth Gate

All four dashboards share `app/dashboard/layout.tsx`, which is the single enforcement point for session validity on the client:

1. Read persisted Zustand session (`localStorage "fitfinder-auth-v2"`)
2. If no session → redirect `/login`
3. If session exists → validate via `GET /api/auth/me`
4. Map role → dashboard → redirect on mismatch
5. Mount realtime hooks → `Socket.IO` connect with handshake token

Role routing map (`lib/mock-users.ts` → `roleToDashboardPath`):

USER → <code>/dashboard/user</code>	OWNER → <code>/dashboard/owner</code>
CLERK → <code>/dashboard/clerk</code>	ADMIN → <code>/dashboard/admin</code>

4. Backend — Project Structure

```
Backend/
├─ prisma/
│  └─ schema.prisma      SOURCE OF TRUTH — 20 models, 8 enums
│  └─ seed.ts            Idempotent demo seed (4 roles + catalog)
│  └─ tsconfig.json
├─ scripts/              Standalone debug probes (not in API runtime)
├─ uploads/              Local media (gitignored)
├─ dist/                 tsc build output
├─ app.ts                Express app + CORS + middleware + mounts
├─ server.ts             Bootstrap: email verify, jobs, listen
├─ index.ts              HTTP + Socket.IO server instance
├─ nodemon.json          watch src, ext ts,json → ts-node
└─ src/
   ├─ config/            (4)  env · database · email · owner-plan catalog
   ├─ routes/            (11) 105 REST endpoints
   ├─ middlewares/       (5)  auth · roles · validate · upload · errorHandler
   ├─ controllers/       (12)
   ├─ services/          (11) domain folders (see §3.2)
   ├─ repositories/      (11) Prisma data access
   ├─ socket/            (1)  init + rooms + emit helpers
   ├─ schema/            (1)  Zod validation schemas
   └─ utils/             (7)  cookies · jwt · hash · responses · revenue
```

Module	Responsibility
app.ts	Composes the Express application: helmet, CORS allowlist with localhost/LAN passthrough in dev, cookie-parser, static /uploads , all 11 route mounts, health check, error handler
server.ts	Async bootstrap — verifies email config, runs the plan-snapshot backfill, starts the 60-second notification/expiry job loop, then listens on env.PORT
config/env.ts	Typed environment accessor with requireEnv / requireSecret . Refuses insecure defaults when NODE_ENV=production ; derives the CORS allowlist from FRONTEND_URL + FRONTEND_URLS
middlewares/auth.ts	Verifies the Bearer JWT, then re-checks the user exists. Returns ACCOUNT_DELETED for removed accounts and rejects CLERK users with no gym assignment — a valid token alone is never sufficient
utils/cookies.ts	Auth cookie policy. Issues SameSite=None; Secure; HttpOnly so the refresh cookie survives the cross-origin frontend → API boundary; maxAge is parsed from the JWT expiry env vars rather than hardcoded

4.1 Scheduled Jobs

server.ts starts a 60-second interval that runs the notification job loop. This is what expires overdue memberships and sends advance expiry reminders without any external cron service:

```
setInterval(() => void runNotificationJobs(), 60_000)
```

runNotificationJobs() responsibilities:

- Expire gym memberships past expiresAt → status = EXPIRED
- Notify members 7 days before expiry → dedupeKey membership:{id}:7d
- Notify owners of pending walk-in approvals
- Notify gym staff of expiring memberships
- Emit notifications_updated over Socket.IO for live badge sync

Idempotency. Reminder notifications use a unique `dedupeKey` on the Notification model. Re-running the job cannot spam a member with duplicate expiry warnings, which is what makes a 60-second poll safe.

5. Frontend — Project Structure

Path	Purpose
lib/api.ts	Axios instance. Attaches the Bearer token on every request; on 401 runs the refresh-retry pipeline; handles ACCOUNT_DELETED with a user-facing notice
lib/socket.ts	Socket.IO singleton with JWT handshake and auto-reconnect
stores/ (29)	Zustand domain stores — auth, gym catalog, members, approvals, sales, messages, notifications, walk-in approvals, create-gym
hooks/ (12)	Realtime sync hooks that bind a socket event to a store refresh
components/ui/	Shared primitives: Button, Input, Label, Checkbox, Alert
app/dashboard/*/components/	Per-role panels (OwnerSidebar, MembersPanel, RevenueChart, CoachesPanel, ...)

5.1 API Client — Refresh Pipeline

The most safety-critical client module. A 401 on any non-auth endpoint triggers exactly one refresh, then replays the original request:

```
Request interceptor:
  read token (memory → localStorage) → Authorization: Bearer <token>

Response interceptor (on error):
  if status ≠ 401                → reject
  if already retried (_retry flag) → reject    (no infinite loop)
  if URL is an auth endpoint      → reject    (no refresh-on-refresh)
  if no stored token              → reject    (logged out)
  if code === ACCOUNT_DELETED     → clear session, notify, → /login?removed=1

  if (!refreshPromise)           ← Promise lock: parallel 401s
    refreshPromise = POST /api/auth/refresh  share ONE refresh call
    → store new access token (memory + localStorage + Zustand)
    → merge returned user/role into session
  finally → refreshPromise = null

  replay original request with the new token
  on refresh failure → clear session → redirect /login
```

Why the Promise lock matters. A dashboard mounts ~12 sync hooks at once. When an access token expires, every in-flight request 401s simultaneously. Without the lock, that becomes a burst of N parallel refresh calls; the server rotates refresh tokens on every use, so all but one would fail and log the user out. The lock collapses the burst into a single refresh.

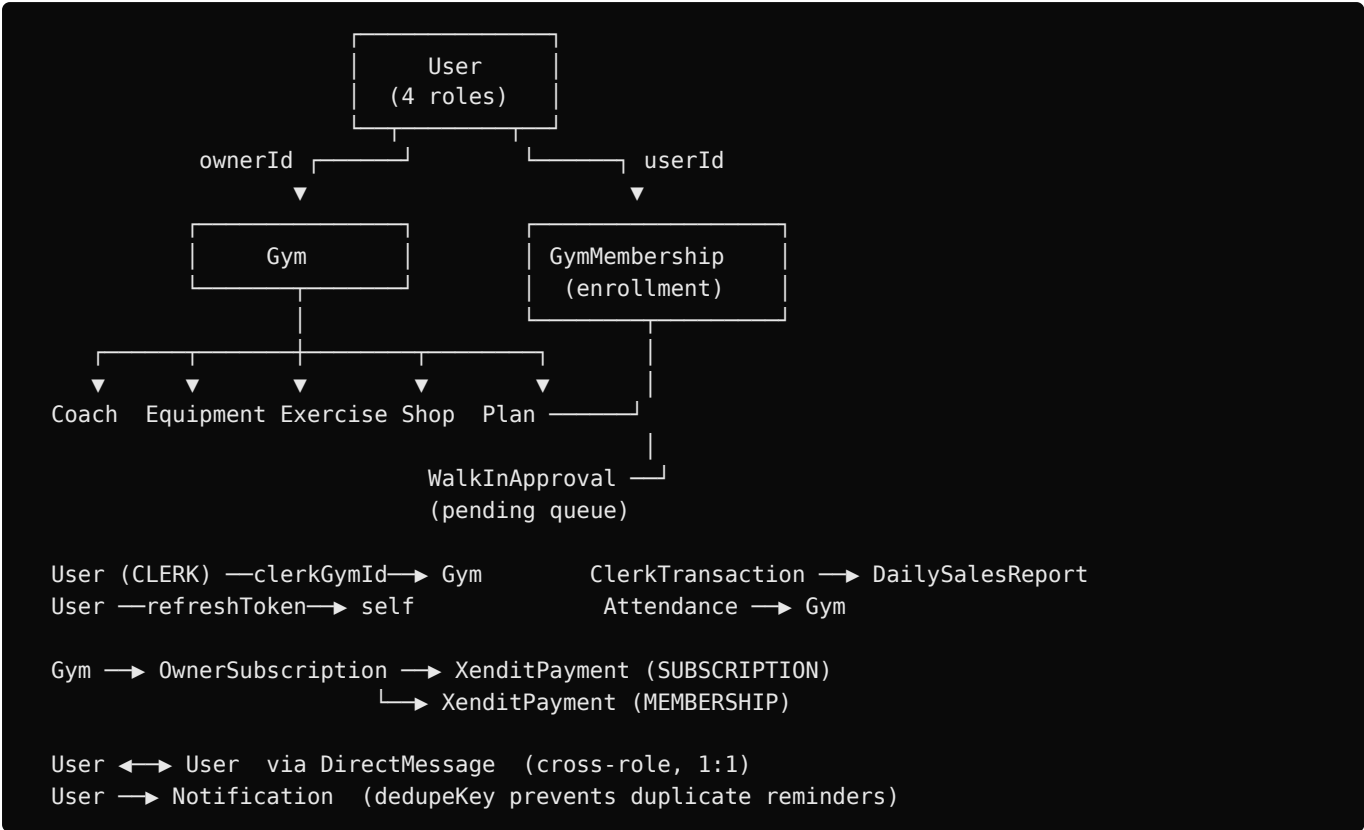
6. Database Design

PostgreSQL via Neon. Schema source of truth: `Backend/prisma/schema.prisma` . 20 models, 8 enums.

6.1 Model Inventory

Table	Purpose	Key Relationships
users	Accounts, roles, email verification, OTP reset state, refresh token, active gym context	Owns gyms, memberships, messages, notifications, attendances
gyms	Gym profile, schedule, pricing, status, optional per-gym Xendit key	Belongs to owner (User); has many plans, coaches, equipment, memberships
membership_plans	Gym-defined SKUs — name, price, durationDays, live subscriber count	Belongs to gym; soft-delete via isActive
gym_memberships	Member enrollments with plan snapshot, payment provenance, period dates	User + Gym + optional Plan + optional Coach
walk_in_approvals	Join/renew approval queue with full snapshot fields	User + Gym + optional Plan
owner_subscriptions	Paid owner platform plan periods (referenceNo unique)	Owner User + optional Gym
xendit_payments	GCash payment records — SUBSCRIPTION or MEMBERSHIP	Belongs to payer User
clerk_transactions	OTC cash / walk-in / renewal sales lines	Gym + Clerk User + optional DailySalesReport
daily_sales_reports	Immutable closed-day summaries with revenue totals	Gym + closing Clerk; has many transactions
attendances	Check-in / check-out log for members and walk-ins	Gym + optional User (null for anonymous walk-ins)
direct_messages	Cross-role 1:1 chat with read receipts	Sender + Receiver Users
direct_conversation_hides	Per-user soft-hide of a DM thread	User, unique on (userId, peerId)
conversations / messages	Legacy gym-scoped thread container (superseded by direct_messages)	Gym; message belongs to sender
notifications	Persistent inbox with dedupeKey, readAt	Belongs to User
coaches	Trainer profiles, specialty, session price, weekly schedule JSON	Belongs to gym
equipment	Inventory with quantity and status	Belongs to gym
exercises	Exercise library with sets/reps/rest, media, form tips	Belongs to gym
shop_products	Gym shop catalog	Belongs to gym
admin_activities	Platform activity feed for the admin dashboard	Standalone

6.2 Core Entity Relationships



6.3 Enums

Enum	Values	Purpose
UserRole	USER, OWNER, CLERK, ADMIN	Authorization tier — drives dashboard routing and API role gates
GymStatus	PENDING, ACTIVE, DECLINED	New gyms await admin approval before appearing publicly
MembershipStatus	ACTIVE, EXPIRING, EXPIRED	Drives access control and renewal prompts
ApprovalStatus	PENDING, APPROVED, DECLINED	Walk-in approval queue lifecycle
PaymentMethod	CASH, CASHLESS, XENDIT	Distinguishes OTC cash from cashless in sales reports
TransactionType	MONTHLY, SESSION, SUPPLEMENTS, DAY_PASS, RENEWAL, COACH	Categorizes individual sales lines
EquipmentStatus	AVAILABLE, IN_USE, UNDER_MAINTENANCE	Gym equipment availability display
ActivityTone	SUCCESS, INFO, WARNING	Visual tone of admin activity feed entries

6.4 Design Decisions Worth Defending

Decision	Reasoning
Plan snapshots on memberships	<code>GymMembership</code> stores <code>planName</code> , <code>planPrice</code> , <code>durationDays</code> copied at purchase time. If an owner renames a plan or soft-deletes it, existing members keep their original terms. Without this, editing catalog data would silently rewrite the legal record of what people paid for.
Soft-delete plans via <code>isActive</code>	Plans are hidden from new purchases but existing memberships are unaffected. Hard deletion would cascade-break member records.
Nullable <code>Attendance.userId</code>	Walk-in visitors may have no account. The row stores <code>memberName</code> and <code>paymentAmount</code> directly so revenue is captured even for anonymous visitors.
Immutable daily sales reports	Once a clerk closes the day, the report and its transactions are a historical record. Later edits to transactions do not retroactively alter a closed report.
<code>dedupeKey</code> on notifications	Unique index makes scheduled reminders idempotent. Essential because the job loop runs every 60 seconds.
Cascade vs SetNull on deletes	Gym → membership is Cascade (a gym cannot exist without its enrollments). User → message is Cascade, but User → clerkGymId is SetNull (deleting a user must not delete the gym).

7. REST API Reference

Base URL `/api` · 105 endpoints across 11 routers · auth via `Authorization: Bearer <accessToken>` unless noted.

7.1 Endpoint Summary by Router

Router	Mount	Count	Purpose
auth	<code>/api/auth</code>	12	Registration, verification, login, refresh, password reset, profile
admin	<code>/api/admin</code>	7	Platform dashboard, users, gyms, analytics, transactions
owner	<code>/api/owner</code>	33	Gym operations, catalog CRUD, staff, reports, settings
clerk	<code>/api/clerk</code>	19	Front desk: transactions, members, approvals, close day
user	<code>/api/user</code>	13	Gymer: join, membership, AI chat, content
messages	<code>/api/messages</code>	6	Cross-role direct messaging
gyms	<code>/api/gyms</code>	5	Public catalog + gym CRUD
notifications	<code>/api/notifications</code>	4	Inbox read state
payments	<code>/api/payments</code>	3	Xendit GCash create / status / webhook
subscriptions	<code>/api/subscriptions</code>	2	Owner platform plan purchase
upload	<code>/api/upload</code>	1	Image upload (multer)

7.2 Authentication Endpoints

Method	Path	Auth	Purpose
POST	<code>/auth/register</code>	RATE	Create account, send 6-digit verification code
POST	<code>/auth/verify-email</code>	RATE	Verify code, set emailVerified
POST	<code>/auth/resend-verification</code>	RATE	Re-issue verification code
POST	<code>/auth/login</code>	RATE	Verify credentials, issue tokens + cookies
POST	<code>/auth/refresh</code>	COOKIE	Rotate token pair from refresh cookie
POST	<code>/auth/forgot-password</code>	RATE	Send reset OTP (enumeration-safe response)
POST	<code>/auth/verify-reset-code</code>	RATE	Verify OTP → issue one-time session token
POST	<code>/auth/reset-password</code>	RATE	Set new password, revoke all refresh tokens
PUT	<code>/auth/change-password</code>	JWT	Authenticated password change
GET	<code>/auth/me</code>	JWT	Current user profile
PUT	<code>/auth/me</code>	JWT	Update fullName / avatarUrl
POST	<code>/auth/logout</code>	JWT	Revoke refresh token, clear cookies

RATE = behind the 30 requests / 15 minutes rate limiter. **JWT** = Bearer token required. **COOKIE** = reads the httpOnly refresh cookie.

7.3 Owner Endpoints (33) — role OWNER

Area	Endpoints
Gym profile	GET /owner/my-gym
Payment settings	GET PUT /owner/payment-settings
Members	GET /owner/members , DELETE /owner/members/:id
Plans	GET POST /owner/membership-plans , PUT DELETE /owner/membership-plans/:id
Coaches	GET POST /owner/coaches , PUT DELETE /owner/coaches/:id
Equipment	GET POST /owner/equipment , PUT /owner/equipment/:id , PUT /owner/equipment/:id/toggle , DELETE /owner/equipment/:id
Exercises	GET POST /owner/exercises , PUT DELETE /owner/exercises/:id
Shop	GET POST /owner/shop , PUT DELETE /owner/shop/:id
Staff	GET POST /owner/staff , DELETE /owner/staff/:id
Reports	GET /owner/sales-reports , GET /owner/sales-reports/:id
Messages	GET POST /owner/messages (legacy gym chat)

7.4 Clerk Endpoints (19) — role CLERK

Area	Endpoints
Dashboard	GET /clerk/dashboard — today's sales, member count, pending approvals
Transactions	GET POST /clerk/transactions
Members	GET /clerk/members , POST /clerk/members (register walk-in)
Plans	GET /clerk/plans
Approvals	GET /clerk/approvals , PUT /clerk/approvals/:id/approve , PUT /clerk/approvals/:id/decline
Walk-in payments	GET /clerk/walk-in-payments , POST /clerk/walk-in-payments/:id/complete
Close day	GET /clerk/sales/closing-preview , POST /clerk/sales/close

7.5 Admin, User, Payments & Messaging

Router	Endpoints
Admin (7)	GET /admin/dashboard , GET /admin/gyms , GET /admin/analytics , GET /admin/users , GET /admin/users/:id , DELETE /admin/users/:id , GET /admin/transactions
User (13)	POST /user/join-gym , GET /user/membership , GET /user/memberships , DELETE /user/membership , POST /user/switch-active-gym , GET /user/walk-in-status , POST /user/walk-in-done/:id , POST /user/ai-chat , GET /user/exercises , GET /user/equipment , GET /user/shop , GET POST /user/messages (legacy)
Payments (3)	POST /payments/create-gcash , GET /payments/:id/status , POST /payments/xendit-webhook WEBHOOK
Messages (6)	GET /messages/search , GET /messages/conversations , GET /messages/thread/:userId , POST /messages/thread/:userId/read , DELETE /messages/conversations/:userId , POST /messages
Notifications (4)	GET /notifications , GET /notifications/unread-count , POST /notifications/read-all , POST /notifications/:id/read

8. Authentication & Security Architecture

8.1 Token Model

Token	Lifetime	Storage	Purpose
Access token	15 minutes	Memory + <code>localStorage</code> + <code>httpOnly</code> cookie	Authorizes every API request via Bearer header
Refresh token	7 days	<code>httpOnly</code> cookie (primary), returned in body	Exchanges for a new access token; rotated on every use

8.2 Login Sequence



8.3 Refresh & Token Rotation

```
Any API call returns 401 (access token expired)
|
▼
Frontend api.ts response interceptor
|
| • not an auth endpoint
| • not already retried
| • session exists
|
▼
POST /api/auth/refresh (cookie sent automatically, withCredentials)
|
▼
Server reads httpOnly refreshToken cookie
|
|─ verify signature + expiry
|─ user still exists?      —no→ 401 ACCOUNT_DELETED
|─ CLERK still assigned to gym? —no→ 401 ACCOUNT_DELETED
|─ stored refreshToken matches? —no→ 401 invalid
|
▼
Issue NEW access + NEW refresh, overwrite stored refreshToken
|
▼
Client persists new token, replays the original request
└─ Promise lock ensures parallel 401s trigger ONE refresh
```

Rotation semantics. Refresh tokens are single-use: each refresh overwrites the stored value. If a refresh token is replayed after it has already been consumed, the stored value no longer matches and the request is rejected. This limits the window in which a stolen refresh token is usable.

8.4 Security Controls

Control	Implementation	Location
Password hashing	bcrypt, 12 rounds	utils/hash.ts
Token hashing	Reset OTPs and session tokens stored as SHA-256 hashes, never plaintext	utils/hash.ts
Brute-force throttling	30 requests / 15 min on login, register, OTP, reset. Deliberately excluded from /refresh and /me so normal sessions never trip it	routes/auth.routes.ts
OTP attempt limiting	Failed reset attempts counted; lockout at PASSWORD_RESET_MAX_ATTEMPTS	services/auth
User enumeration defence	forgot-password returns an identical response whether or not the email exists	services/auth
HTTP security headers	helmet, with cross-origin resource policy relaxed for image serving	app.ts
CORS allowlist	Explicit origin set from env; localhost/LAN regex passthrough in development only	app.ts , config/env.ts
Input validation	Zod schemas parsed in middleware before controllers run	schema/
Stale-token rejection	JWT alone is insufficient — authenticate re-queries the user and rejects deleted accounts and unassigned clerks	middlewares/auth.ts
Cookie hardening	httpOnly, Secure, SameSite=None so JS cannot read the refresh token	utils/cookies.ts
Production secret enforcement	requireSecret throws in production if a JWT secret is missing or matches the dev default	config/env.ts
Webhook verification	Xendit callbacks validated against a configured webhook token	controllers/payment

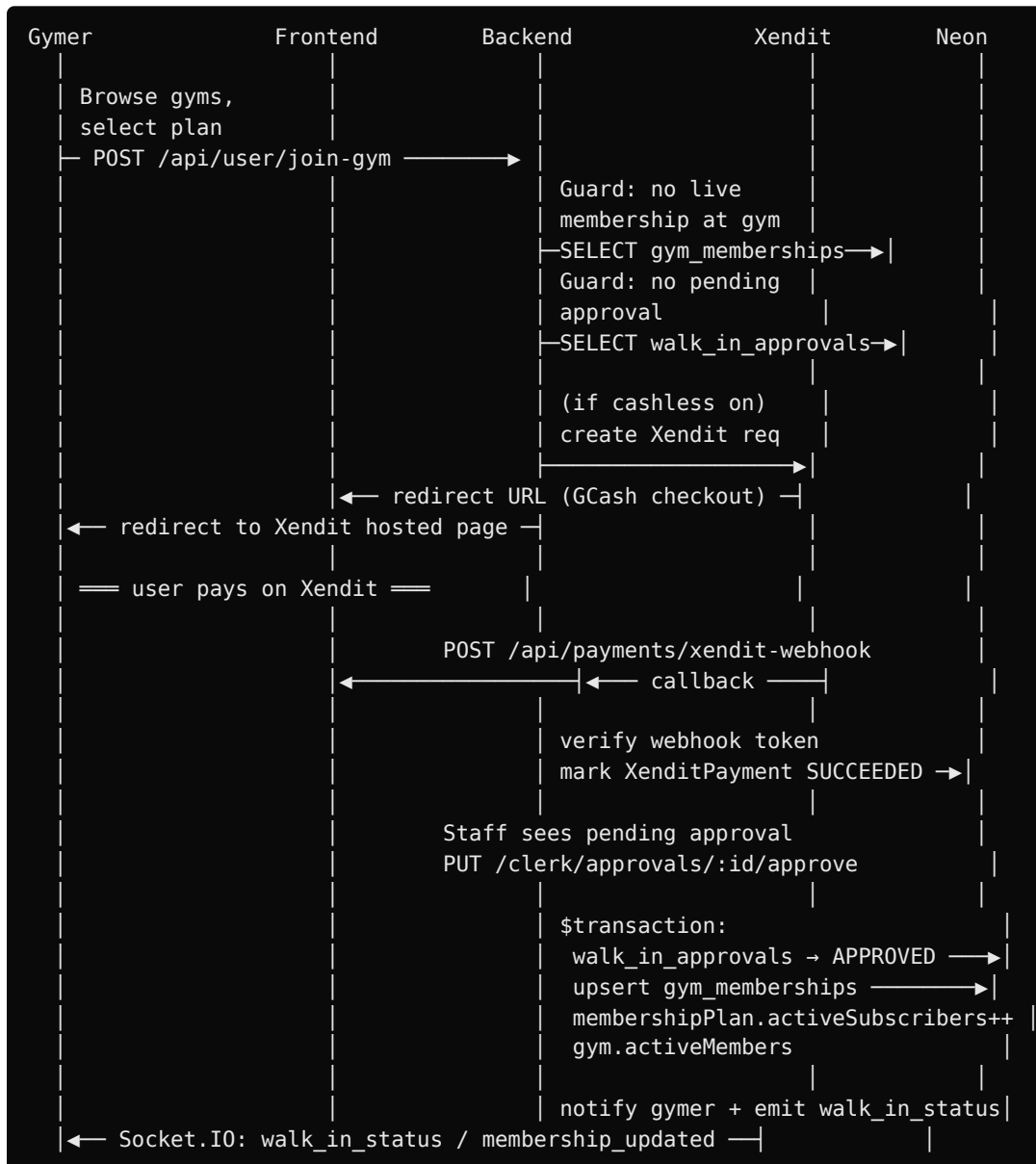
Cross-origin cookie requirement. The frontend (port 3000 / fitfinder.fun) and API (port 5000 / Render) are different origins. A cookie with SameSite=Lax is withheld by the browser on cross-origin XHR, which silently breaks refresh. SameSite=None is the only value permitting cross-site delivery, and browsers require Secure alongside it — hence auth requires HTTPS end-to-end.

9. Membership & Payment Flow

9.1 Two Membership Channels

	Walk-in (OTC)	Online (GCash)
Initiated by	Clerk at the front desk	Gymer from the dashboard
Payment	Cash, recorded immediately	Xendit GCash, confirmed by webhook
Approval	Created already APPROVED	Created PENDING → staff approve
memberType	WALK_IN	ONLINE
paymentMethod	CASH	XENDIT
registeredBy	OWNER or CLERK	SELF
Activation	Immediate on record	On approval after payment confirmation

9.2 Online Purchase → Activation



9.3 Renewal

Renewal reuses the approval queue with `isRenewal: true`. On approval the service reads the existing membership's `startsAt` / `expiresAt` and extends the period rather than creating a duplicate row, so a member never has two concurrent enrollments at the same gym.

9.4 Membership Expiry

```
Every 60s: runNotificationJobs()
|
| — SELECT gym_memberships WHERE expiresAt < now AND status ≠ EXPIRED
|   → set status = EXPIRED (bulk updateMany)
|
| — memberships expiring in exactly 7 days
|   → notify member (dedupeKey membership:{id}:7d) ← prevents duplicates
|
| — emit notifications_updated to affected users
```

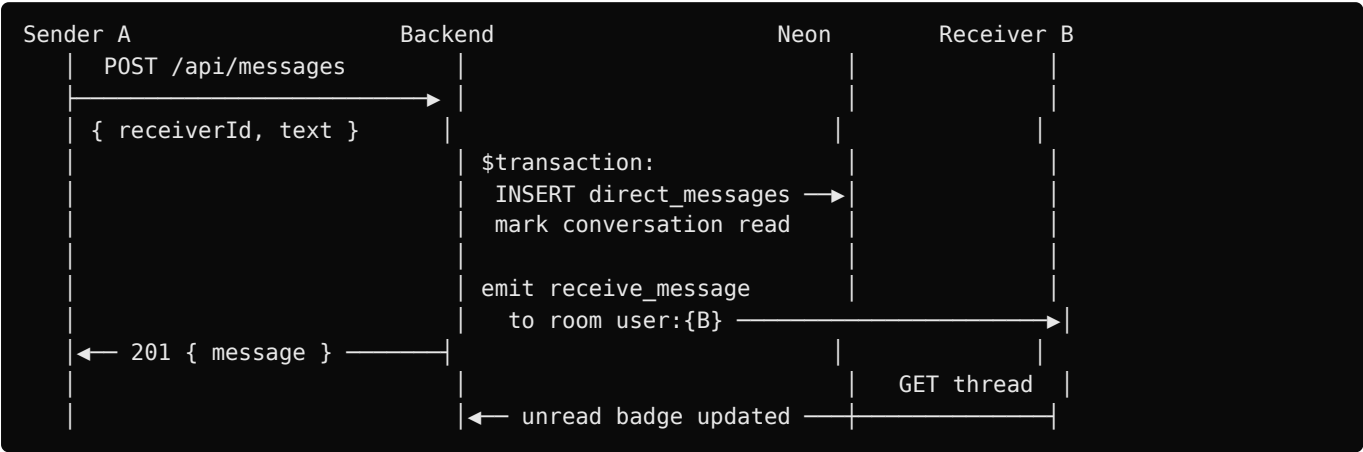
9.5 Revenue Model

Two distinct revenue streams — a common point of confusion. Platform revenue (Admin dashboard) is computed from `xendit_payments` where `type = 'SUBSCRIPTION'` and `status = 'SUCCEEDED'` — these are owner platform-plan purchases. Gym-side OTC revenue lives in `clerk_transactions` and `daily_sales_reports` and does **not** feed platform revenue. The admin dashboard shows the platform's own earnings; the owner dashboard shows the gym's.

10. Messaging Flow

FitFinder supports cross-role 1:1 direct messaging — a gymer can message a gym owner, a clerk, or an admin through the same interface.

10.1 Send & Receive



10.2 Soft-Hide Semantics

Hiding a conversation writes a `direct_conversation_hides` row rather than deleting messages. The hidden thread disappears for the actor only, and remains fully intact for the peer — who receives `conversation_deleted` so their UI updates too. The unique index on `(userId, peerId)` makes re-hiding idempotent.

10.3 Legacy Threads

The `conversations` / `messages` tables implement an earlier gym-scoped chat. These endpoints remain mounted for backward compatibility (`/owner/messages` , `/user/messages`), while `direct_messages` serves all new traffic. Both are documented as legacy in the database reference.

11. Real-Time Layer (Socket.IO)

11.1 Connection & Rooms

```
Client connect
|
| handshake.auth.token = <accessToken JWT>
▼
Server verifies JWT —invalid→ disconnect
|
▼
Auto-join rooms:
• user:{userId}    personal – DMs, notifications, account events
• gym_catalog      global gym list changes

Client events:
• join_gym(gymId)  → join gym:{gymId}
• leave_gym(gymId) → leave gym:{gymId}

Server fan-out helpers (socket/index.ts):
emitToUser(userId, event, payload) → room user:{id}
emitToGym(gymId, event, payload)   → room gym:{id}
emitToRoom(room, event, payload)
```

11.2 Event Catalogue

Event	Direction	Purpose
join_gym / leave_gym	C → S	Subscribe/unsubscribe from gym-scoped updates
receive_message	S → C	New direct message delivered live
conversation_deleted	S → C	Thread hidden by peer
conversation_hidden	S → C	Hide confirmation for the actor
notification	S → C	New inbox notification
notifications_updated	S → C	Unread count changed
notification_read	S → C	Single item marked read elsewhere
walk_in_status	S → C	Gymer's join/renew approved or declined
walk_in_approvals_updated	S → C	Staff approvals queue refreshed
membership_updated	S → C	Member's enrollment changed
members_updated	S → C	Owner/clerk member list refreshed
sales_updated	S → C	Sales and reports refreshed
equipment_updated	S → C	Equipment inventory changed
shop_updated	S → C	Shop catalog changed
coaches_updated	S → C	Coach roster changed
membership_plans_updated	S → C	Gym plan catalog changed
gym_plans_catalog_updated	S → C	Global catalog — home page starting prices
attendance_updated	S → C	Check-in/out changed the Active Now list
active_gym_changed	S → C	Multi-gym member switched context
admin_gyms_updated	S → C	Admin gym list or revenue changed
admin_users_updated	S → C	Admin user list changed
owner_gym_cleared	S → C	Owner's gym deleted — clear local state
owner_must_repurchase	S → C	Owner plan lapsed — force repurchase
account_deleted	S → C	Account removed while online — force logout

11.3 Sync Hook Pattern

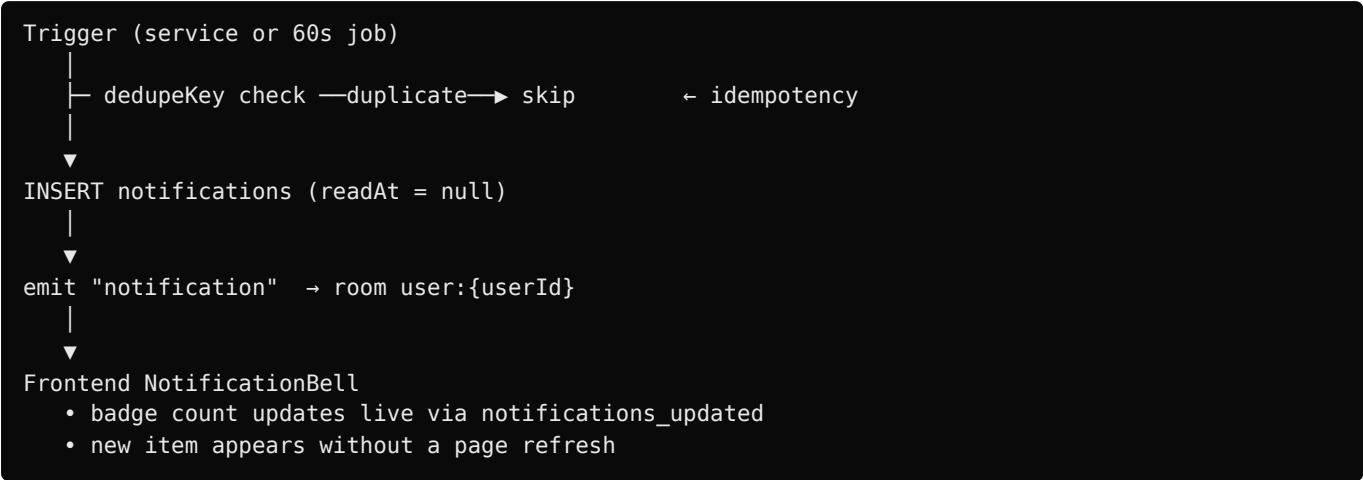
Each frontend hook binds one socket event to one store refresh, keeping realtime concerns out of components:

```
useSalesSync.ts      sales_updated      → salesStore.refresh()
useMembersListSync.ts members_updated    → membersStore.refresh()
useWalkInApprovalSync.ts walk_in_approvals_updated → approvalsStore.refresh()
useAdminRealtimeSync.ts admin_users_updated → adminStore.refresh()
useMemberGymContentSync equipment/shop_updated → contentStore.refresh()
useDirectMessageSocket receive_message    → messageStore.append()
useNotificationsSocket notification                → notificationStore.prepend()
```

Why push instead of polling. The approval queue, sales figures, and Active Now list all change as a result of actions taken by *other* users. Polling would either feel laggy or generate constant load. Socket.IO rooms give targeted delivery to exactly the users whose view is affected.

12. Notification System

12.1 Delivery Pipeline



12.2 Notification Producers

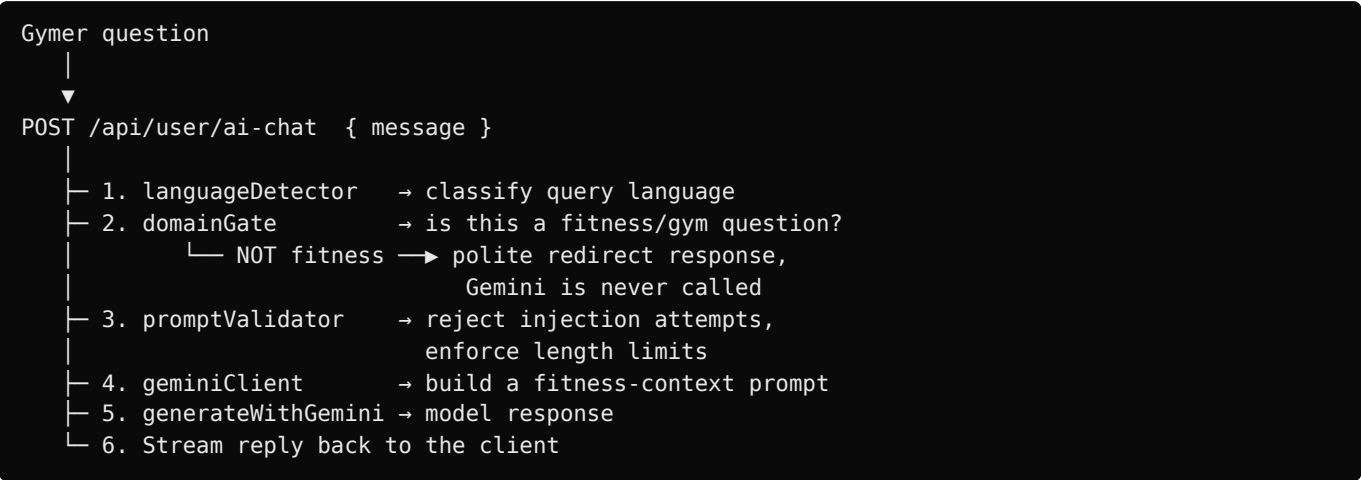
Producer	Recipients	Trigger
notifyMembershipRequestSubmitted	Gym owner + clerks	Gymer submits a join/renew request
notifyMembershipApproved	Gymer	Staff approve the request
notifyMembershipRejected	Gymer	Staff decline, with reason
notifyMembershipExpired	Gymer	Job loop detects expiry (7-day warning first)
notifyGymStaff	Owner + clerks of a gym	Expiring memberships, new approvals
notifyMembershipChange	Gymer	Any membership mutation
upsertLiveNotification	Target user	Live countdowns (e.g. walk-in onboarding)

12.3 Why dedupeKey Matters

The job loop runs every 60 seconds. Without a unique dedupe key, a "7 days before expiry" reminder would be inserted 1,440 times per day per member. The unique index on `Notification.dedupeKey` makes the insert fail silently on subsequent runs, so reminders fire exactly once.

13. AI Fitness Assistant

13.1 Request Pipeline



13.2 Guard Layers

Layer	Purpose
languageDetector.ts	Identifies the query language so the assistant can respond in kind — relevant for a Filipino user base using Taglish
Domain gate	Confines the assistant to fitness and gym topics. This is both a product decision and a cost control: irrelevant queries never reach the API
promptValidator.ts	Sanitizes input and blocks prompt-injection patterns before the message is embedded in a system prompt
geminiClient.ts	Isolates the SDK behind one function so the provider can be swapped via <code>AI_PROVIDER</code> without touching business logic

Privacy note. Conversation history is client-side only and is not persisted to Neon. Chat transcripts are not stored server-side, which avoids retaining user health-adjacent data.

14. Attendance & Reporting

14.1 Attendance Model

Manual check-in/check-out for both registered members and anonymous walk-ins. An open attendance (no checkedOutAt) means the person is currently in the gym.

Member check-in

1. Resolve the clerk's gym (user.clerkGymId, or role default)
2. Guard: an open attendance must not already exist for this user
3. Verify an ACTIVE membership exists at that gym
4. INSERT attendances { gymId, userId, type: MEMBER, paymentAmount: 0 }

Walk-in check-in

1. Resolve the clerk's gym
2. INSERT attendances { gymId, userId: null, memberName, type: WALK_IN, paymentAmount }
3. INSERT clerk_transactions (CASH sale line)

Checkout

```
UPDATE attendances SET checkedOutAt = now()
```

14.2 Active Now

```
GET /clerk/dashboard → countActiveNow(gymId)
```

```
countActiveNow = attendances.count({ gymId, checkedOutAt: null })
```

Owner view additionally groups by gym:

```
prisma.attendance.groupBy({ by: ['gymId'], where: { checkedOutAt: null } })
→ per-gym Active Now counts across all owned gyms
```

14.3 Clerk Daily Close-Out

```
GET /clerk/sales/closing-preview
```

→ today's open transactions + totals, nothing persisted yet

POST /clerk/sales/close

```
$transaction:
```

1. Re-verify no other clerk closed the day
(prevents two clerks closing the same date)
2. CREATE daily_sales_reports
 { gymId, clerkId, closedByRole, reportDate (@db.Date),
 totalTransactions, totalRevenue, closedAt }
3. UPDATE clerk_transactions SET dailySalesReportId
 → transactions are now sealed into the report

Owner receipt:

```
GET /owner/sales-reports/:id → the immutable closed report
```

Multi-gym owners. A gym owner can operate several gyms. Owner dashboards aggregate across them by first collecting the owner's gym IDs, then querying memberships and attendance scoped to that set — visible in the owner overview where per-gym Active Now counts are grouped.

15. Role-Based Access Control

15.1 Role Capabilities

Capability	Admin	Owner	Clerk	Gymer
Browse gym catalog	✓	✓	✓	✓
Join a gym / buy membership	—	—	—	✓
Register walk-in member	—	✓	✓	—
Approve / decline requests	—	✓	✓	—
Manage plans / coaches / equipment / shop	—	✓	✓	—
Check-in / check-out members	—	✓	✓	—
Close the sales day	—	✓	✓	—
View sales reports	—	✓	✓	—
Configure Xendit API key	—	✓	—	—
Add / remove staff	—	✓	—	—
Manage all users / remove accounts	✓	—	—	—
Approve / decline gyms	✓	—	—	—
Platform analytics & revenue	✓	—	—	—
AI fitness assistant	—	—	—	✓
Direct messaging (any role)	✓	✓	✓	✓

15.2 Defence in Depth

Authorization is enforced at four independent layers, so a mistake in any single one is not sufficient to breach access:

1. CLIENT `dashboard/layout.tsx` routes by role – UX only, never trusted as a security boundary
2. ROUTE `requireRole('OWNER')` middleware on the router chain
3. CONTROLLER Ownership scoping in the query itself – e.g. an owner update filters by `{ id, ownerId: req.userId }`, so a forged gym ID cannot modify another owner's gym
4. SERVICE Business rules such as CLERK must have `clerkGymId` before any staff operation succeeds

Layer 3 is the important one. Role middleware answers "may an OWNER call this endpoint?" Data scoping answers "may *this* OWNER modify *this specific* record?" Both are required. Without scoping, any owner who guessed or discovered another gym's ID could edit it.

15.3 Account Deletion Cascade

Admin account removal runs a dedicated `purgeUserRecords(tx, userId)` inside a single transaction, explicitly deleting dependent rows and disassociating references before the user row is removed. The affected user

receives an `account_deleted` socket event that forces an immediate logout, and any in-flight request returns `ACCOUNT_DELETED` rather than a generic 401.

16. Deployment & DevOps

16.1 Environments

	Local Development	Production
Frontend	<code>next dev --turbo</code> (port 3000)	Vercel / static build (fitfinder.fun)
Backend	<code>nodemon</code> → <code>ts-node src/server.ts</code> (port 5000)	Render web service (free plan)
Database	Neon PostgreSQL — same managed instance, pooled connections	
Hot reload	<code>nodemon</code> watches <code>src</code> for <code>.ts/.json</code> ; Next.js Turbopack for frontend	—

16.2 Render Blueprint

```
services:
  - type: web
    name: fitfinder-api
    runtime: node
    rootDir: Backend
    plan: free
    buildCommand: npm install --include=dev && npx prisma generate && npm run build
    startCommand: node dist/index.js
    healthCheckPath: /api/health

    envVars:
      NODE_ENV=production          FRONTEND_URL=https://www.fitfinder.fun
      NODE_VERSION=20              FRONTEND_URLS=www + apex
      DATABASE_URL                 [sync: false – set in dashboard]
      JWT_ACCESS_SECRET            [generateValue: true]
      JWT_REFRESH_SECRET           [generateValue: true]
      JWT_ACCESS_EXPIRES_IN=15m
      JWT_REFRESH_EXPIRES_IN=7d
      RESEND_API_KEY               [sync: false]
      XENDIT_SECRET_KEY            [sync: false]
      GEMINI_API_KEY               [sync: false]
```

16.3 Configuration Management

All configuration flows through a typed accessor in `config/env.ts` with two tiers:

```
requireEnv(key, fallback?)      missing + no fallback → throw at boot
requireSecret(key, devFallback)
  └ value present and ≠ devFallback → use it
  └ NODE_ENV=production             → THROW (never boot with a dev secret)
  └ development                     → warn loudly, use the fallback

Derived at boot:
  ALLOWED_FRONTEND_ORIGINS = dedupe([FRONTEND_URLS, FRONTEND_URL].split(","))
                          ?? ["http://localhost:3000"]
```

16.4 Secrets Handling

Secret	Storage	Delivery
DATABASE_URL	Backend/.env (gitignored)	Render dashboard, sync: false
JWT_*_SECRET	Generated by Render	Blueprint generateValue — never committed
XENDIT_SECRET_KEY	Dashboard + optional per-gym DB column	Never returned to member-facing responses
GEMINI_API_KEY , RESEND_API_KEY	Dashboard	sync: false

.env.example contains placeholders only and is safe to commit. .env is gitignored at both the repo root and in Backend/ .

17. Design Decisions & Trade-offs

17.1 Decisions Worth Defending

Decision	Alternative rejected	Reasoning
Stateless JWT access tokens	Server-side sessions	Scales without a shared session store and survives restarts on Render's free tier, which cycles instances. Cost: revocation is not instant, mitigated by the DB re-check in <code>authenticate</code>
15-minute access token + silent refresh	Long-lived access token	A stolen access token is usable for 15 minutes rather than 8 hours, while the user still never experiences a logout because refresh is transparent
Refresh token rotation	Reusable refresh token	Single-use tokens limit the replay window for a stolen credential
Socket.IO rooms over REST polling	Interval polling	Approval queues and sales figures change from other users' actions; polling is laggy and wasteful
Plan snapshots on memberships	Live relation only	Catalog edits must never retroactively alter what existing members paid for
Webhook-driven payment confirmation	Client-side success redirect	Trusting the client's redirect would let a user activate a membership without paying
Domain-gated AI assistant	General-purpose assistant	Keeps the feature coherent and avoids paying for irrelevant inference
Managed Neon Postgres	Self-hosted Postgres	Removes operational burden; pooled connections suit an intermittent free-tier service that idles
Zod at the middleware boundary	Validation inside controllers	Controllers receive typed, already-valid data and stay focused on business logic

17.2 Acknowledged Limitations

Limitation	Impact	Potential resolution
Access token in <code>localStorage</code>	Readable by injected script on XSS	Move to a memory-only reference with the <code>httpOnly</code> cookie as the sole durable store — the cookie is already the primary mechanism
No automated test suite	Regressions found manually	Unit tests on the auth service and payment state machine; integration tests on the approval flow
Legacy chat tables retained	Two messaging models coexist	Complete migration to <code>direct_messages</code> , then drop <code>conversations / messages</code>
Repository layer not yet wired	Controllers still call Prisma directly	Introduced as a complete data-access boundary; adoption is incremental so behaviour stays verifiable at each step
Free-tier hosting	Cold starts, instance cycling	Paid tier for consistent latency; no architectural blocker
60-second polling for expiry	Up to 60s lag before a member appears expired	Database-native scheduled jobs if the interval becomes noticeable

18. Testing & Verification

18.1 Current State

Verification to date has been static and manual rather than automated:

Method	Coverage	Result
TypeScript compilation	Backend — full project	<code>tsc --noEmit</code> passes with zero errors
Manual integration walkthrough	All four role dashboards	Verified in browser during development
Idempotent seed	Database	<code>prisma/seed.ts</code> uses upsert throughout — safe to re-run
Build verification	Backend	<code>tsc</code> produces <code>dist/</code> consumed by <code>node dist/index.js</code>
Automated unit / integration tests	—	Not implemented

18.2 Recommended Test Priorities

1. **Auth service unit tests** — registration, verification expiry, OTP attempt lockout, refresh rotation, revocation on reset. Highest security value per line written.
2. **Membership state machine** — the transitions between PENDING → APPROVED → ACTIVE → EXPIRED, and that an approval writes atomically.
3. **Payment webhook** — duplicate webhook delivery must be idempotent; a replayed callback must not double-activate.
4. **Authorization scoping** — assert that an owner cannot read or mutate another owner's gym, plans, or members by ID.
5. **Job idempotency** — running `runNotificationJobs()` repeatedly must not create duplicate reminders.

18.3 Demo Accounts

Provided by `prisma/seed.ts` for evaluation. All use password `12345` with `emailVerified: true`.

Role	Email	Notes
Platform Admin	<code>admin@test.com</code>	Full platform access, revenue analytics
Gym Owner	<code>owner@test.com</code>	Owns 4 seeded gyms; plan "Standard" active
Front Desk Clerk	<code>clerk@test.com</code>	Assigned to gym-seed-1
Gymer	<code>gymer@test.com</code>	Active membership on the 3-month Pro plan

Seed data includes 4 gyms (one PENDING to demonstrate admin approval), 3 membership plans, 2 coaches, 4 equipment items, 3 exercises, 4 shop products, an active membership, a closed owner subscription, and 5 admin activity entries.

Appendix A — Source Reference

Purpose	Path
Database source of truth	Backend/prisma/schema.prisma
Demo seed data	Backend/prisma/seed.ts
Express composition	Backend/src/app.ts
Server bootstrap + jobs	Backend/src/server.ts
Environment config	Backend/src/config/env.ts
JWT verification middleware	Backend/src/middlewares/auth.ts
Auth cookie policy	Backend/src/utils/cookies.ts
Auth business logic	Backend/src/services/auth/auth.service.ts
Membership lifecycle	Backend/src/services/membership/
Payment integration	Backend/src/services/payment/xendit.service.ts
Socket.IO rooms	Backend/src/socket/index.ts
Realtime fan-out helpers	Backend/src/services/realtime/realtime.service.ts
Data access layer	Backend/src/repositories/ (11)
API client + refresh pipeline	Frontend/src/lib/api.ts
Socket.IO client	Frontend/src/lib/socket.ts
Dashboard auth gate	Frontend/src/app/dashboard/layout.tsx
Deployment blueprint	render.yaml

Appendix B — Document Notes

B.1 Documentation Drift Corrected

Earlier revisions of the repository documentation referenced `Backend/src/middleware/` (singular) and `src/index.ts` as the composition root. The live codebase uses `middlewares/` (plural) and splits composition across `app.ts` (Express app) and `server.ts` (bootstrap). This document reflects the current structure.

B.2 Repository Layer Status

The backend includes a complete data-access layer of 11 repository classes covering all 20 Prisma models. Every method accepts an optional transaction client so services can compose atomic multi-table writes. At the time of writing, controllers and services still call Prisma directly; repository adoption is incremental by design so that behaviour can be verified at each step rather than in one large change.

B.3 Source Documentation

Narrative documentation also exists as Markdown in `docs/` (`CODE_MAP`, `ROUTES_REFERENCE`, `DATABASE_REFERENCE`, `SOCKET_REFERENCE`, `AUTH_FLOW`, `PAYMENT_FLOW`, `MESSAGE_FLOW`, `AI_ASSISTANT_FLOW`) with `PROJECT_INDEX.md` at the repository root as the navigation entry point. This handbook consolidates that material and adds architectural rationale.